COMP2001: ARTIFICIAL INTELLIGENCE METHODS – LAB 4 EXERCISES

RECOMMENDED TIMESCALES

This lab exercise is designed to take no longer than two hours, and it is recommended that you complete this within a week. The contents of this exercise were covered in lecture 4.

GETTING SUPPORT

The computing exercises have been designed to facilitate flexible studying and can be worked through in your own time or during timetabled lab hours. It is recommended that you attend the labs, even if you have completed the exercises in your own time, to discuss your solutions and understanding with one of the lab support team and/or your peers. It is entirely possible that you might make a mistake in your solution which leads to a false observation and understanding.

OBJECTIVES

The purpose of this lab exercise is to gain experience in implementing a move acceptance method that were covered in lecture 4 and with the parameter setting sensitivity issues of such methods across multiple problem instances.

The topics that will be covered in this exercise are:

- COMP2001 framework background including solution memory.
- Implementation of Simulated Annealing (SA) using two different cooling schedules – Geometric Cooling and Lundy and Mees's Cooling.
- Parameter tuning of geometric cooling (single parameter) using a trial-and-error approach.
- Parameter tuning of Lundy and Mees cooling (two parameters) using a fractional design.

LEARNING OUTCOMES

By completing this lab exercise, you should meet the following learning outcomes:

- Students should gain experience implementing some move acceptance methods for solving combinatorial optimisation problems.
- Students should understand the parameter setting issues and the impact that poor parameter choices have on the performance of such move acceptance methods and metaheuristics more generally.

TASK 1 – IMPORTING LAB EXERCISE FILES

Download the source code for this lab exercise from the Moodle page. You should find various files related to simulated annealing and its cooling schedules, the exercise runner configurations, and the exercise runners themselves.

Drag the `simulatedannealing` folder into your IDE so that it is a direct subfolder of `"com.aim.metaheuristics.singlepoint"`.

The `"Lab4ExercisesTestFrameConfig"` and `"Lab4ExercisesRunner"` classes should be placed within the `"runners"` package alongside the ones from previous labs.

TASK 2 – READING: FRAMEWORK BACKGROUND (REMINDER FROM LAB 3)

For a single point-based search method, the COMP2001 Framework maintains two solutions in memory:

- The solution that we are operating upon (the current solution), and
- A backup of the solution that we started the current iteration with (the backup solution) so that we can reuse it later if we "reject" any modification(s) made to the current solution.

When a `SATHeuristic` is invoked on a problem object, it modifies the solution in the current solution index (`CURRENT_SOLUTION_INDEX`) leaving the candidate solution (s') in the `CURRENT_SOLUTION_INDEX` and does not modify the backup solution in the `BACKUP_SOLUTION_INDEX` within the intermediate solution memory state. The move should then either be accepted ($s_{i+1} \leftarrow s_i'$) or rejected ($s_{i+1} \leftarrow s_i$) using the respective move acceptance criterion.

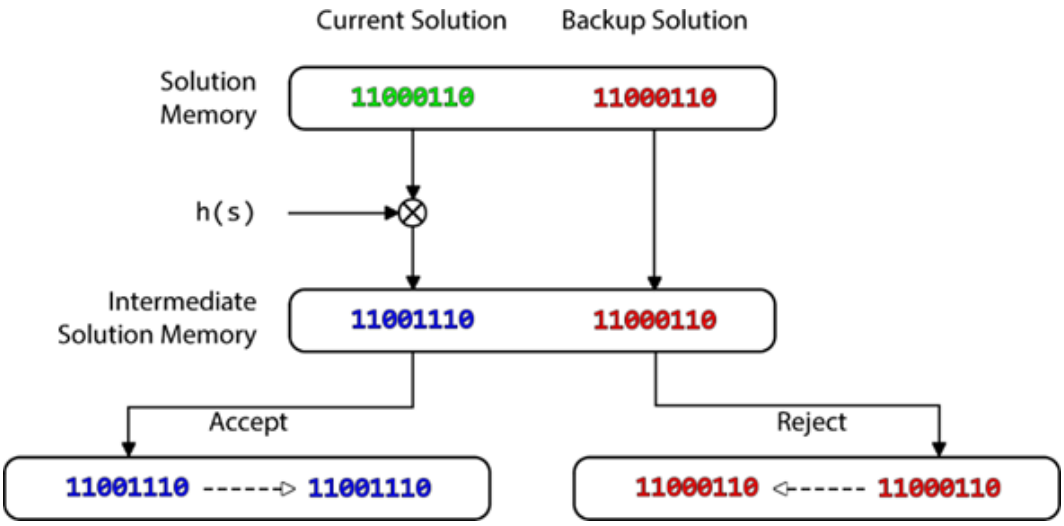

Figure 1 - Single Point-based Search in the COMP2001 Framework. The dashed arrow represents a solution being copied between solution memory indices using the copySolution(int source_memory_index, int destination_memory_index) method of the SAT problem.

TASK 3 – SIMULATED ANNEALING [IMPLEMENTATION]

There are three Classes in the package `com.aim.metaheuristics.singlepoint.simulatedannealing`: `SimulatedAnnealing`, `GeometricCooling`, and `LundyAndMeesCooling`. In each of

these classes, you will need to complete the implementation of Simulated Annealing, and the two respective cooling schedules. Remember that your implementation should perform a **single iteration** of the respective heuristic method, hence no `while(terminationCriterionNotMet)` loop is required.

Running of your solutions is handled by the `Lab4ExercisesRunner` Class that is provided for you, and the experimental parameters can be configured in `Lab4ExercisesTestFrameConfig`. Note also that `Lab4ExercisesRunner` executes the experiments in parallel to reduce the time you need to wait (which is ok to do given an evaluation-based termination criterion such as that used within this framework). Do not be concerned that experiments complete quicker than expected. Your computer may slow down during the experiments due to the high CPU usage and parallelisation. This is normal and is nothing to worry about.

Consider the question following this paragraph. Depending on your exact implementation and reflection to the answer to question 1, these experiments could finish fairly quickly or could take much longer. Discuss your solution if you think it is taking too long.

***Question to think about before implementing Simulated Annealing**. In Java, deep copying of objects is expensive as we need to create a copy of all classes, member variables/fields, and construct a new Object to contain these. The COMP2001 framework performs a deep copy of a solution when using the copySolution(int iIndexFrom, int iIndexTo) API method. How might we reduce the runtime of an algorithm that only uses a bit flip neighbourhood operator?*

SIMULATED ANNEALING

Simulated Annealing (SA) was covered in the lecture on *Move Acceptance in Local Search Metaheuristics and Parameter Setting Issues*. Below is the pseudocode from the lecture. Note that updating of the cooling schedule is purposely abstract. This allows us to use different cooling schedules without having to reimplement the main body of SA. Remember that the test framework handles solution initialisation and the outer loop of the search method. Hence only lines 7 to 14 need to be implemented in the `runMainLoop()` method.

```
INPUTS: { T_0, "any other parameters of the cooling schedule" }
s_0 = generateInitialSolution()
Temp ← T_0
s* ← s_0
s ← s_0
REPEAT
    s' ← perturb(s)
    Δ = f(s') - f(s)
    r ← random ∈ [0,1)
    IF Δ < 0 || r < P(Δ,Temp) THEN
        s ← s'
    FI
```

```
    s* ← updateBest()
    Temp ← updateTemperature()
UNTIL ( termination conditions are met )
return s*
```

You should re-implement the random bit flip heuristic move operator within the implementation of Simulated Annealing. This has the advantage that you know which bit has been flipped in the solution. Remember that at the end of each main loop, in the case of the pseudocode that is after line 15, the solutions in the CURRENT and BACKUP solution indices must be the same in accordance with the COMP2001 Framework specification. (See Figure 1). It is up to you *how* this is done, but efficient implementations will result in faster executions!

IMPORTANT: When generating a random number, whilst there are multiple methods to generate random numbers in the range $[0,1)$, please **only use the nextDouble() method of the random number generator**. Usage of any other method is likely to result in an output different to the expected output (which is fine, but you cannot then verify your implementations likely correctness).

GEOMETRIC COOLING

You should implement the `advanceTemperature()` method to set the '*currentTemperature*' by applying the Geometric Cooling as shown in the below pseudocode. **Make sure that you also set the value of α "appropriately" from within the constructor. You may want to run some experiments once implemented to find a reasonable setting.**

LUNDY AND MEES'S COOLING

You should implement the `advanceTemperature()` method to set the '*currentTemperature*' by applying the Lundy and Mees's cooling as shown in the below pseudocode. **Make sure that you also set the value of β "appropriately" from within the constructor. You may want to run some experiments once implemented to find a reasonable setting.**

TASK 4 – FORMATIVE ASSESSMENT (MOODLE QUIZ)

The questions in the Lab 4 Formative Assessment (link here) require you to perform a number of experiments using your implementation of Simulated Annealing and associated cooling schedules. Answer the questions by configuring the `Lab4ExercisesRunner` and `Lab4TestFrameConfig` classes as appropriate and updating the settings of each cooling schedule from within their classes, running the algorithms, and analysing the resulting outputs. You may want to keep a copy of the results presented after running each of those experiments.

Note that there are no open text box questions in this week's formative quiz. If you would like additional feedback on your answers, please come and speak to one of us in the lab!

ADDITIONAL INFORMATION AND TIPS

How do I: perform a bit flip; get the number of variables; evaluate the solution; etc.

See the COMP2001 Framework API for framework specific questions on Moodle!

How do I use Euler's number in Java?

Java's Math API includes a constant `E` which is Euler's number. The Math API contains a function `exp(double a)` which allows you to calculate *e* to the power of `a`.

E.g. `Math.exp(0) = 1`.